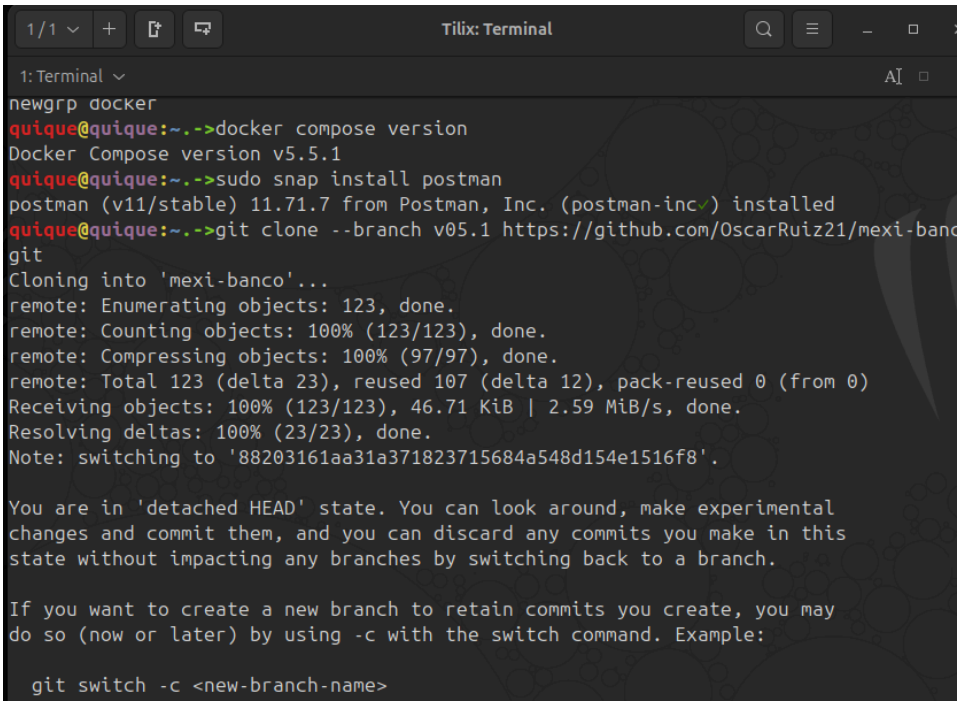
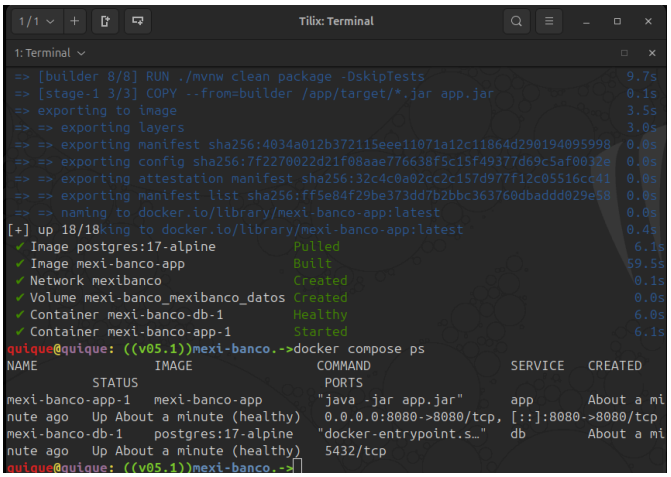
Laboratorio 5 - Sistemas Distribuidos
Medrano Solano Enrique
320152841

Lo primero que se realizó dentro de este laboratorio fue la instalación pertinente de los pre requisitos, en este caso el docker compose y la instalación de postman.

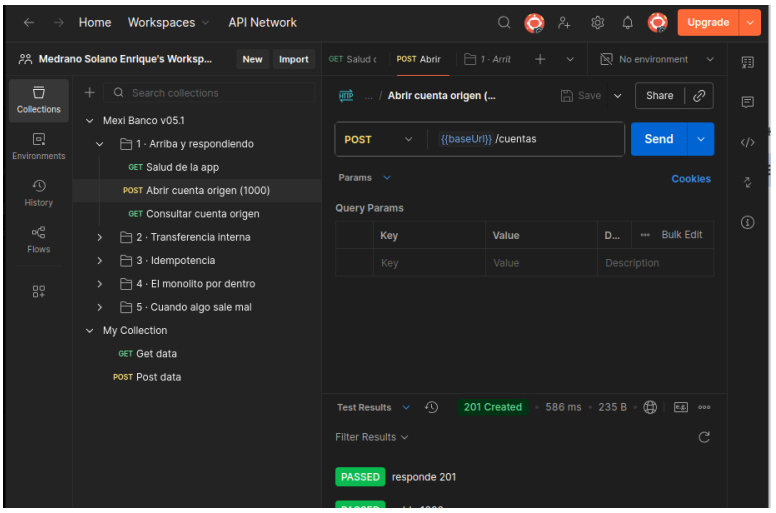


Después de esto se inicializa y prende el docker compose, a su vez que se importa el JSON que el profesor compartió para el Postman.

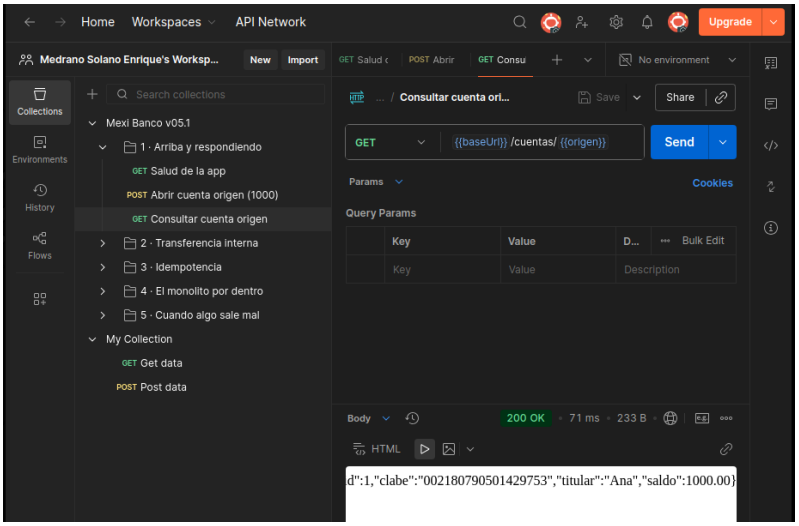


Laboratorio 5 - Sistemas Distribuidos
Medrano Solano Enrique
320152841

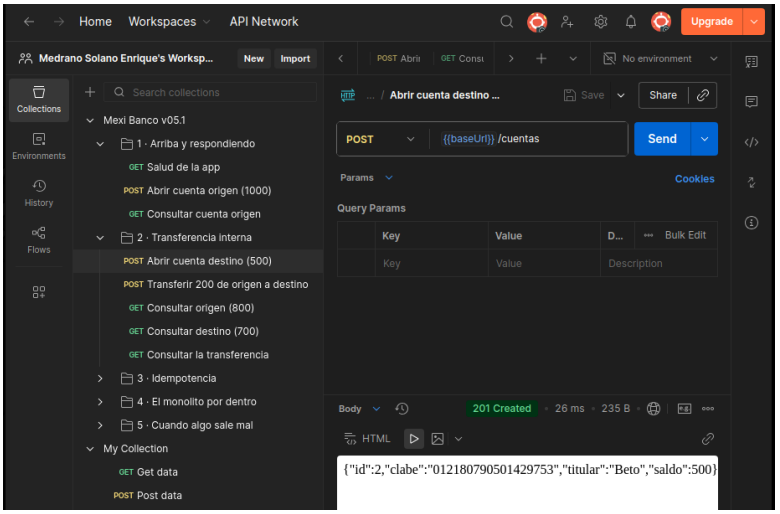
Aquí se comprueba la petición de abrir cuenta origen con 1000 pesos. En donde se hace conexión con el servidor y acepta la petición.



Aquí se hace la consulta a la cuenta origen para saber de quien es cual es su cuenta cable y el saldo disponible.

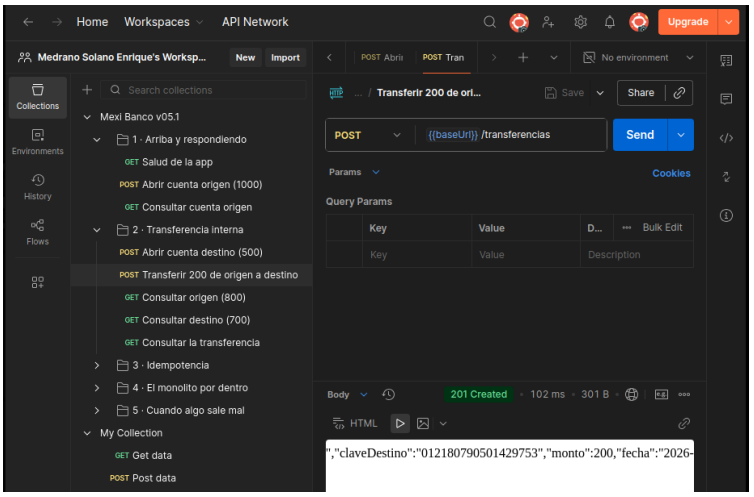


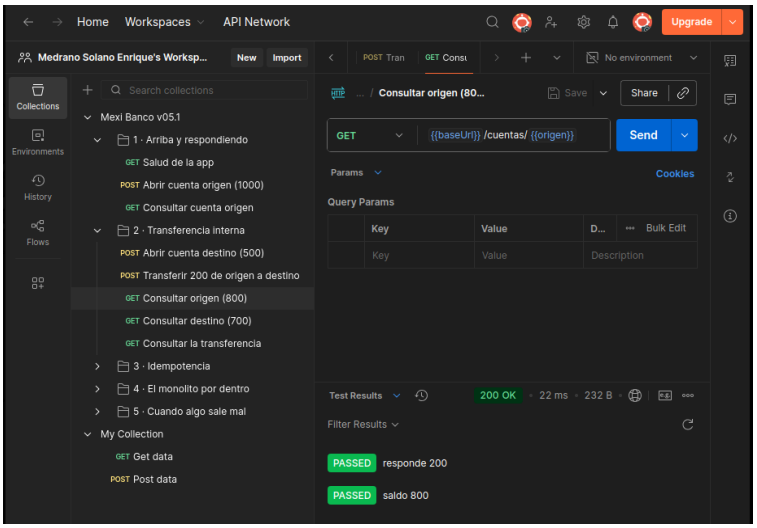
Se hace la petición para la creación de la cuenta destino y poder hacer transacciones más adelante.

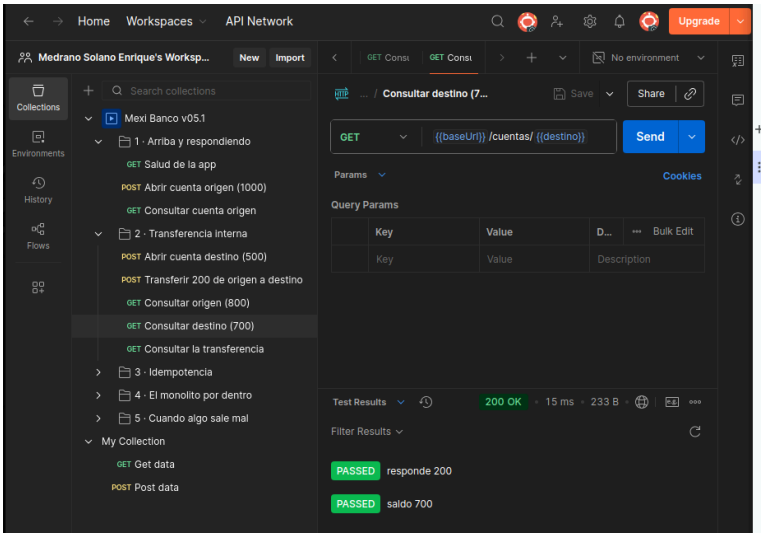


Laboratorio 5 - Sistemas Distribuidos
Medrano Solano Enrique
320152841

En las siguientes capturas lo que se le pide al servidor es una petición sobre la transferencia de 200 pesos de la cuenta origen a la cuenta destino, y a su vez se consulta el saldo de la cuenta origen donde ya marcará 800 pesos, y lo mismo con la destino que ahora tendrá 700 pesos. y Finalmente consultar la transferencia con éxito entre las cuentas.

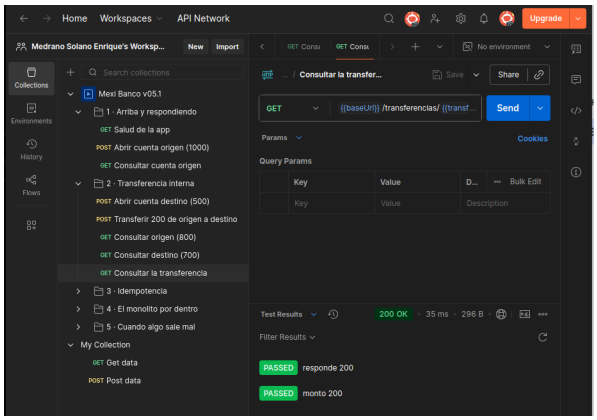


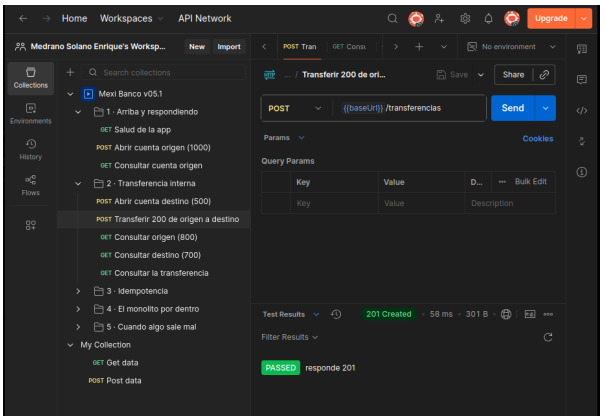


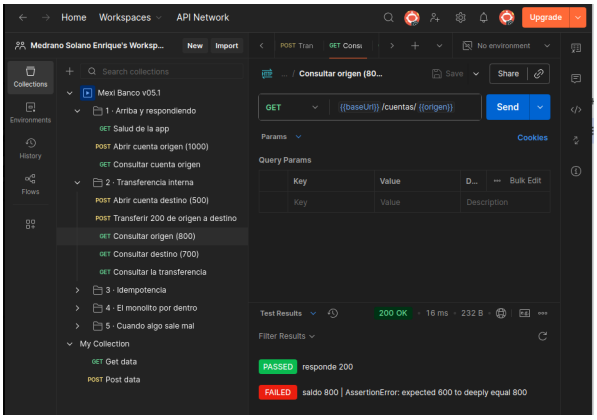


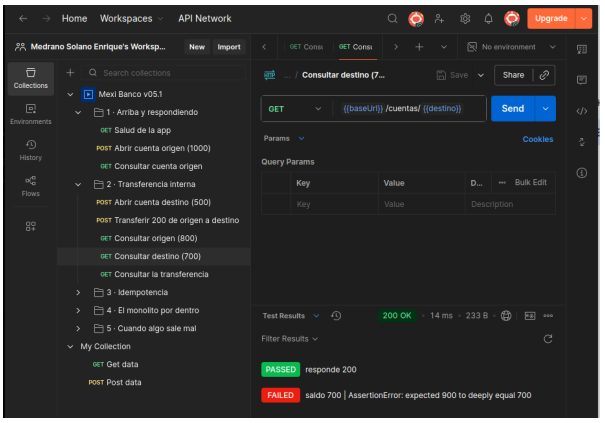
Laboratorio 5 - Sistemas Distribuidos
Medrano Solano Enrique
320152841

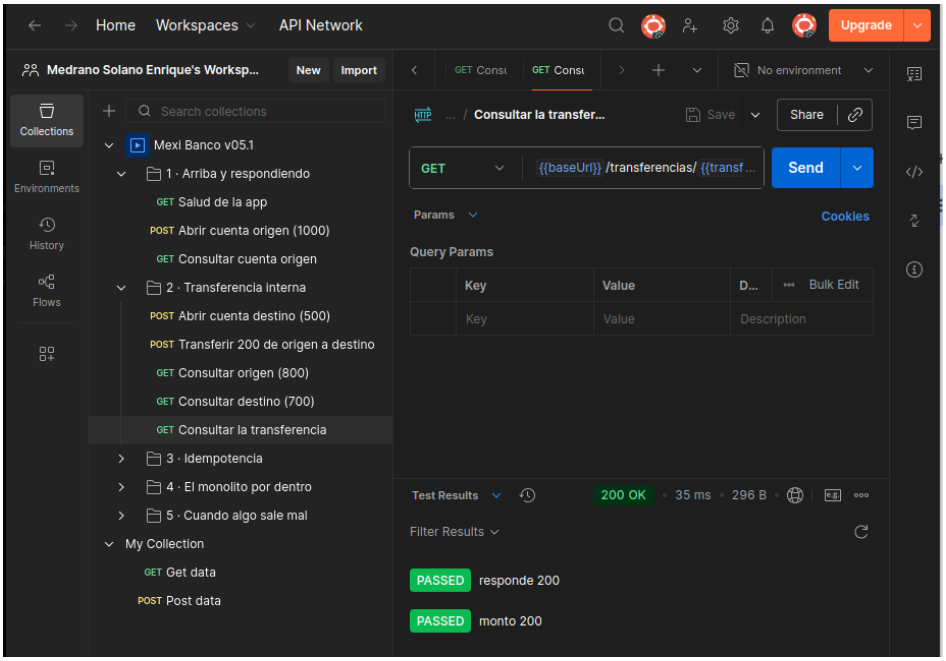
Si se vuelve a realizar la transferencia otra vez, no pasará nada, ya que el sistema es robusto para ese tipo de casos y sucede lo que se muestra en las capturas siguientes con los errores pertinentes, por un duplicado de SPEI.





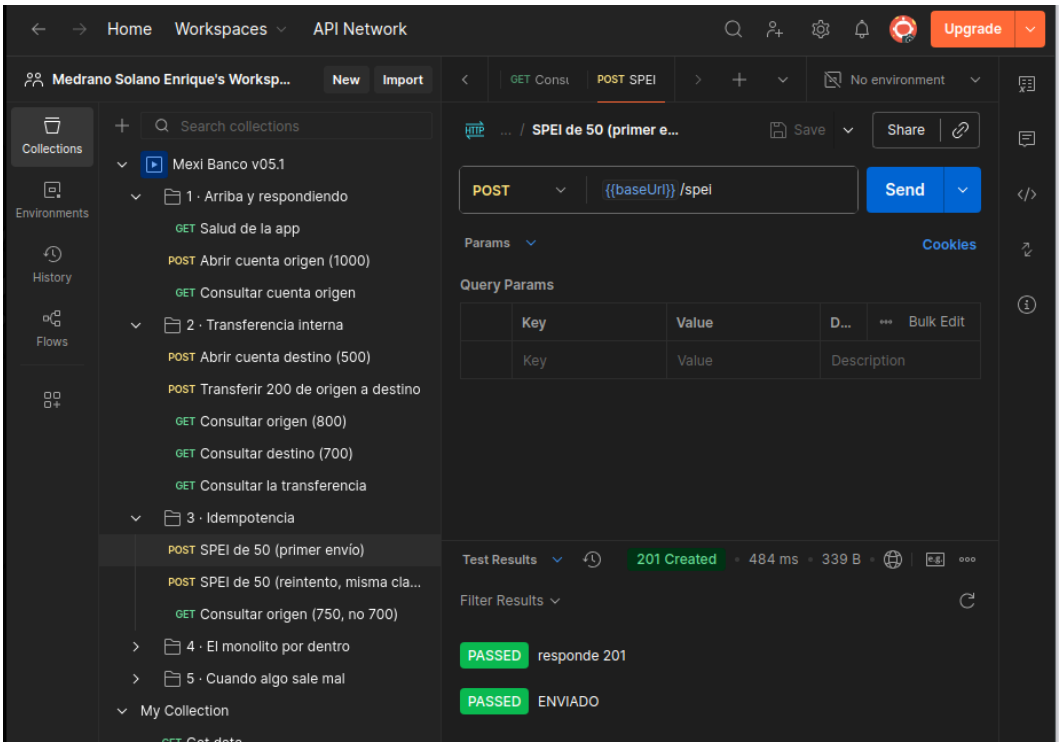


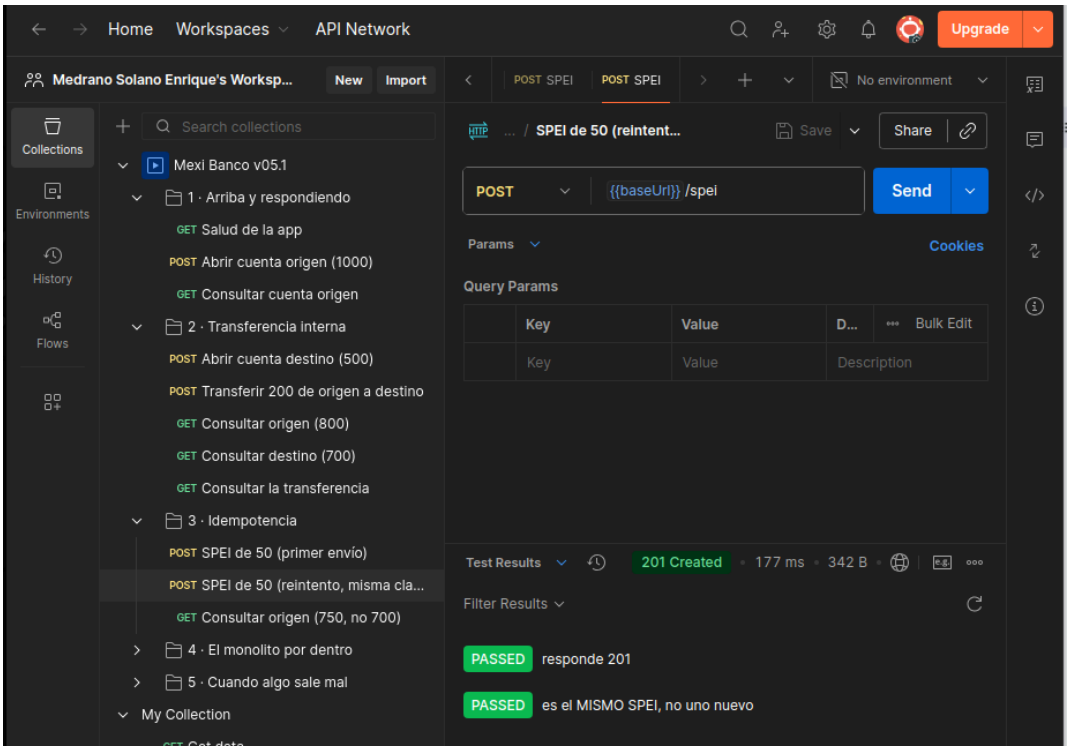




Laboratorio 5 - Sistemas Distribuidos
Medrano Solano Enrique
320152841

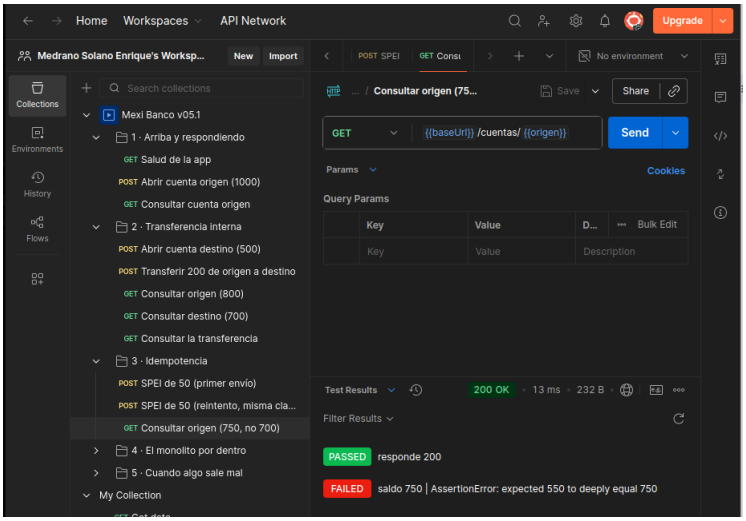
Ahora se procede con una solicitud de SPEI con 50 pesos, y otro intento posterior por la misma cantidad. Pero el sistema también cuenta con la aparte de idempotencia que permite no generar la misma transacción.

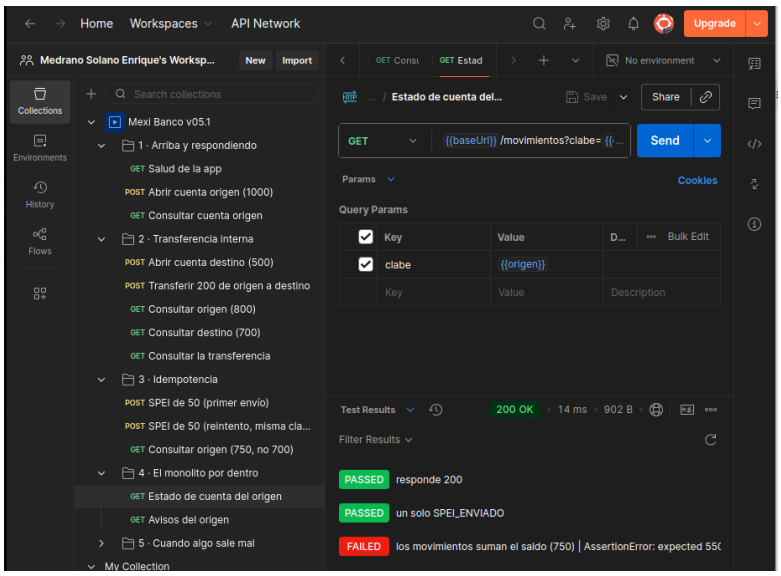


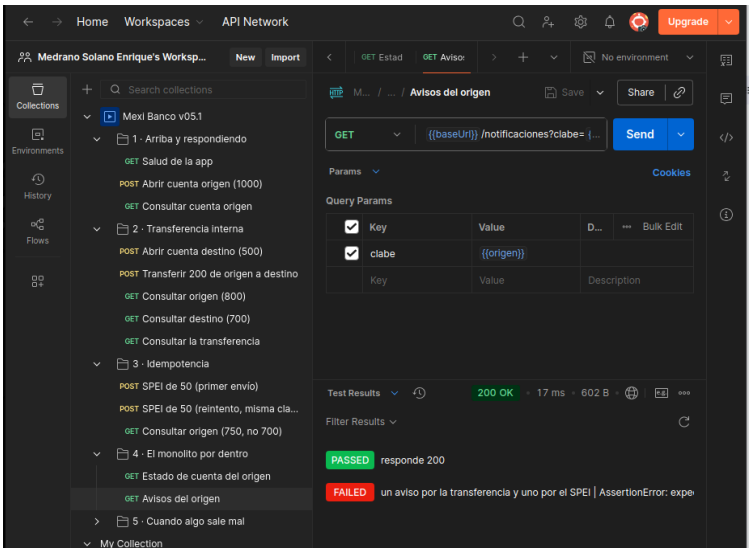


Laboratorio 5 - Sistemas Distribuidos
Medrano Solano Enrique
320152841

Al correr la colección en orden sin duplicar transferencias manualmente, el saldo final refleja únicamente las operaciones válidas enviadas, manteniendo la consistencia con el estado de cuenta.

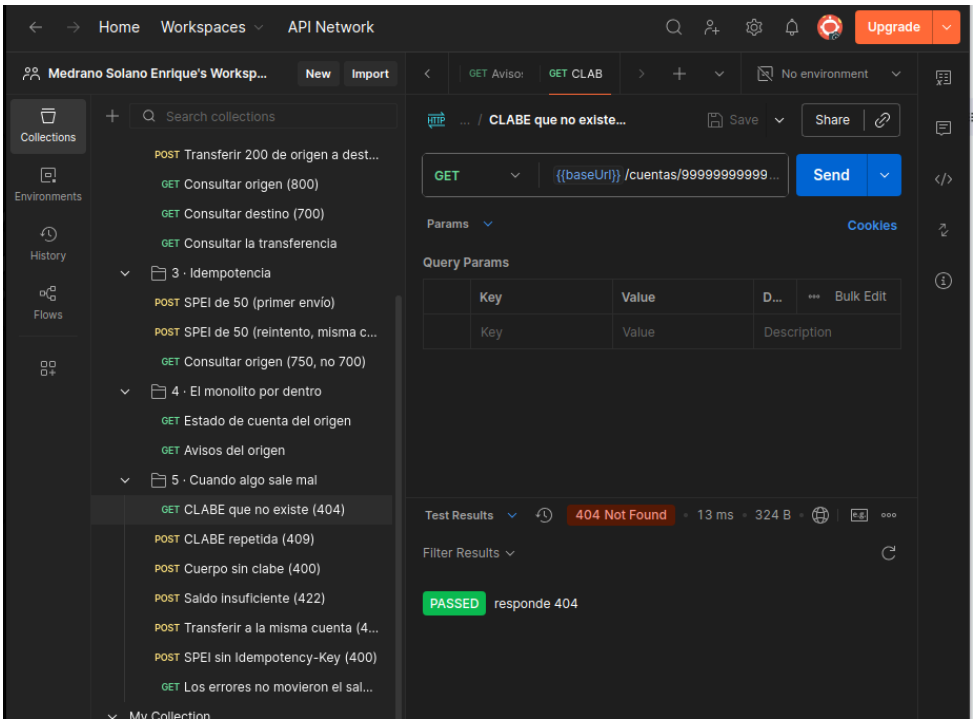


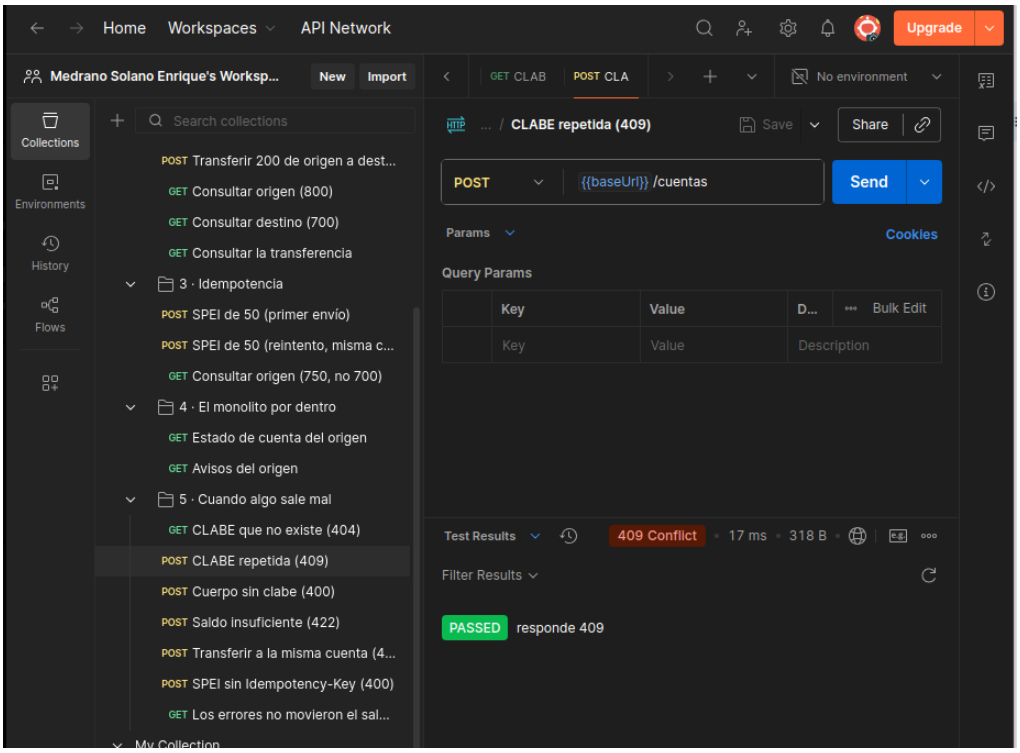




Laboratorio 5 - Sistemas Distribuidos
Medrano Solano Enrique
320152841

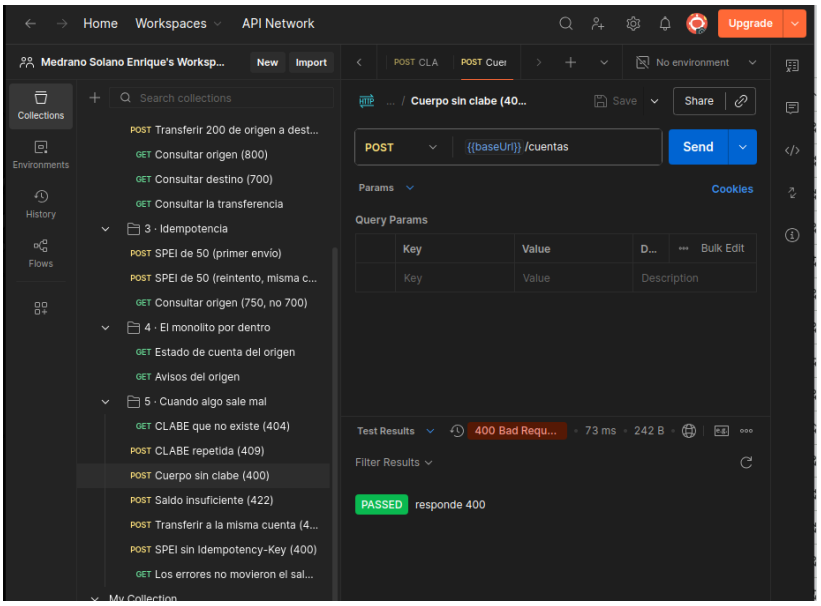
El sistema arroja *Not Found* porque la prueba consulta intencionalmente una CLABE inventada que no está registrada en la base de datos.

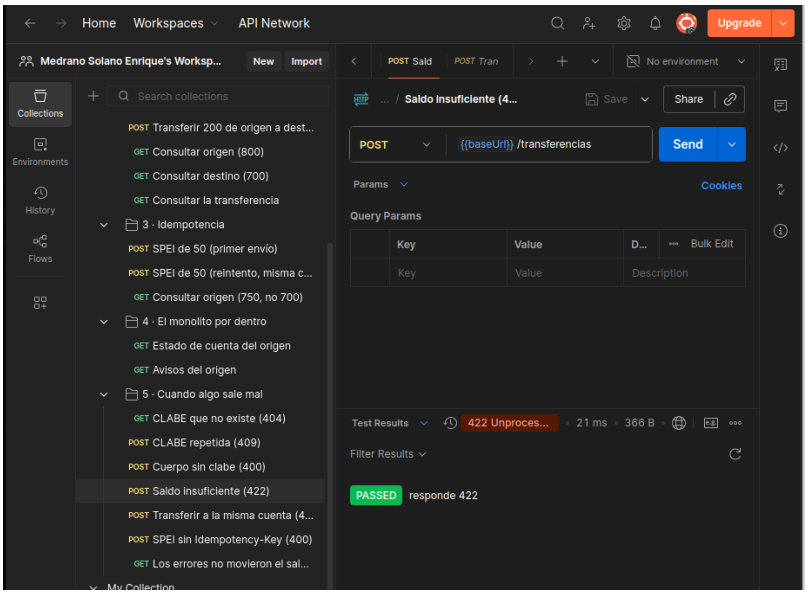




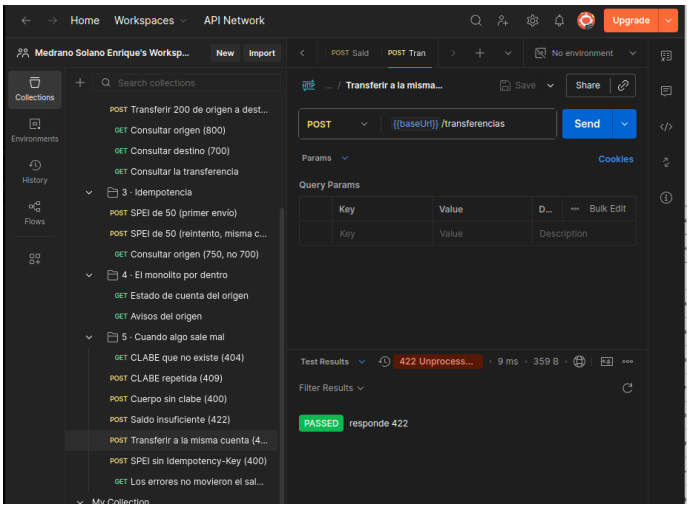
Laboratorio 5 - Sistemas Distribuidos
Medrano Solano Enrique
320152841

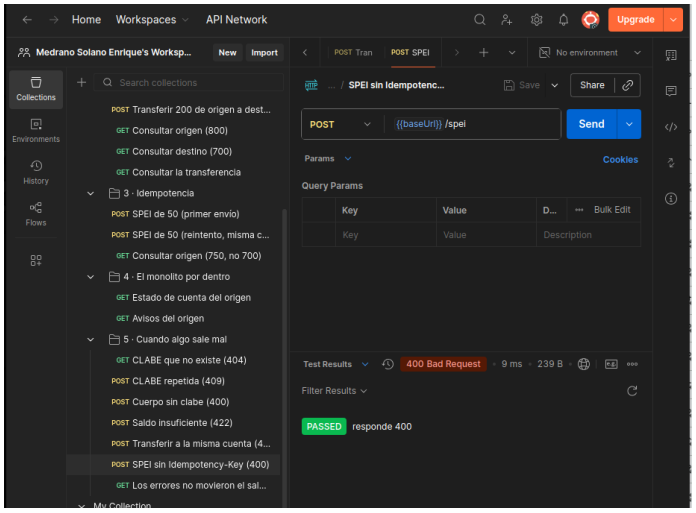
El sistema arroja *Conflict* porque la prueba intenta registrar intencionalmente una cuenta con una CLABE que ya existe, validando que el servidor protege la regla de unicidad.
El sistema arroja *Bad Request* porque la petición está mal formada desde el cliente, omitiendo un campo que es obligatorio para el servidor.
El sistema arroja *Unprocessable Entity* porque la cuenta origen no tiene fondos suficientes para completar la transacción.

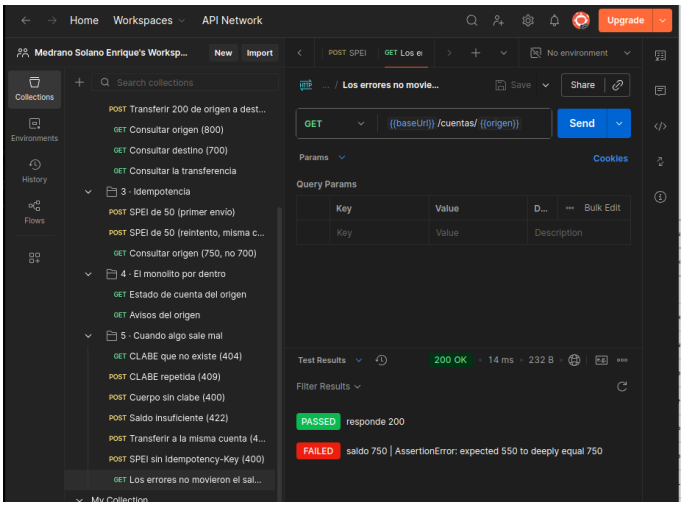




Laboratorio 5 - Sistemas Distribuidos
Medrano Solano Enrique
320152841

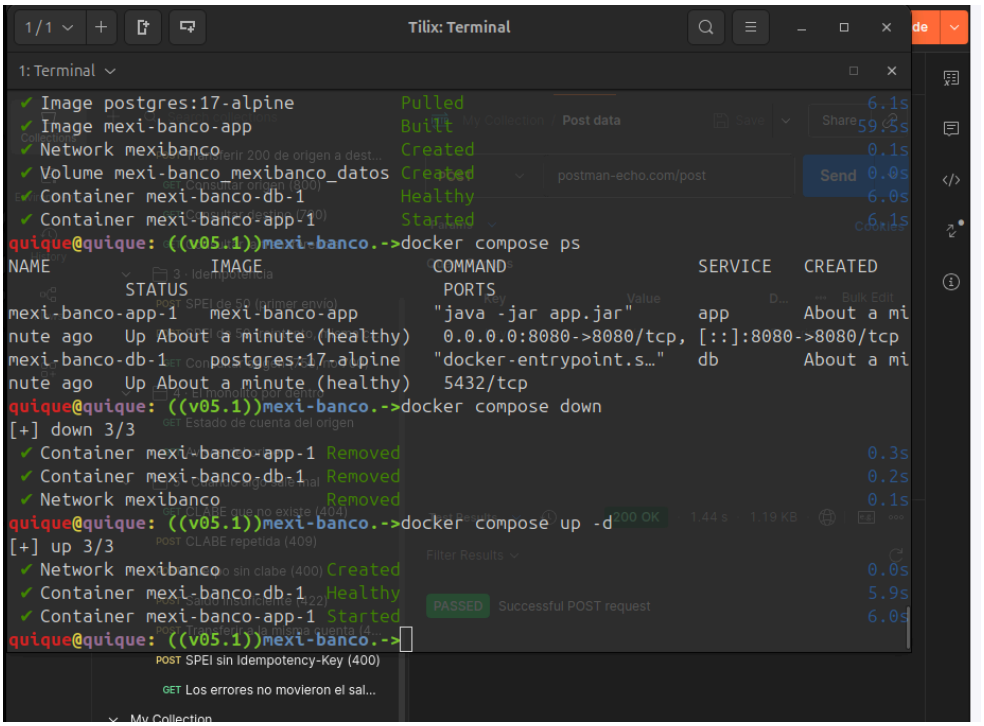




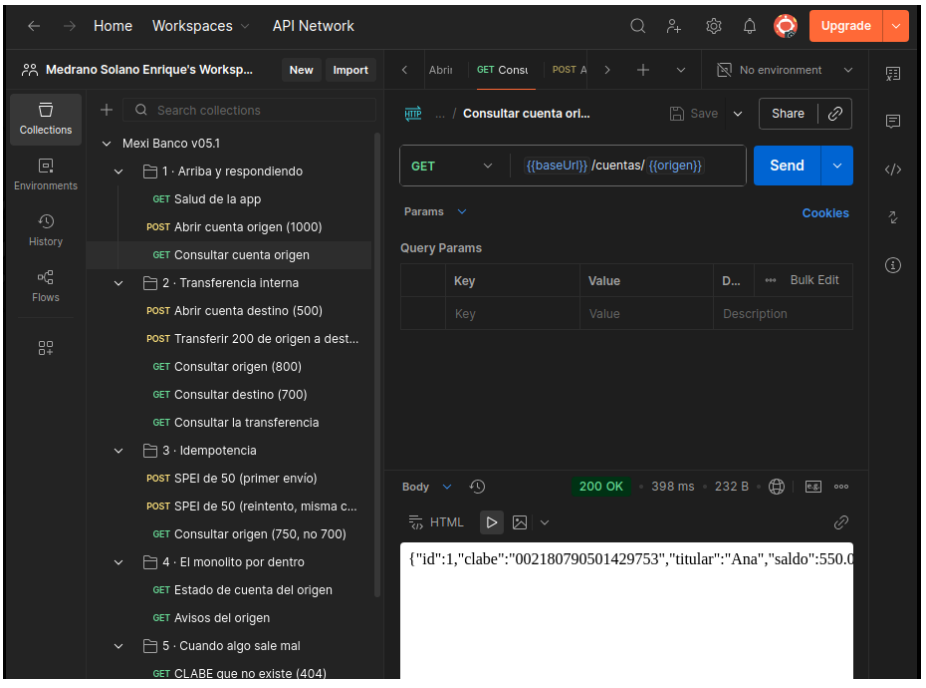


Se apaga el servidor y se vuelve a comprobar si los datos siguen correctos.

Laboratorio 5 - Sistemas Distribuidos
Medrano Solano Enrique
320152841

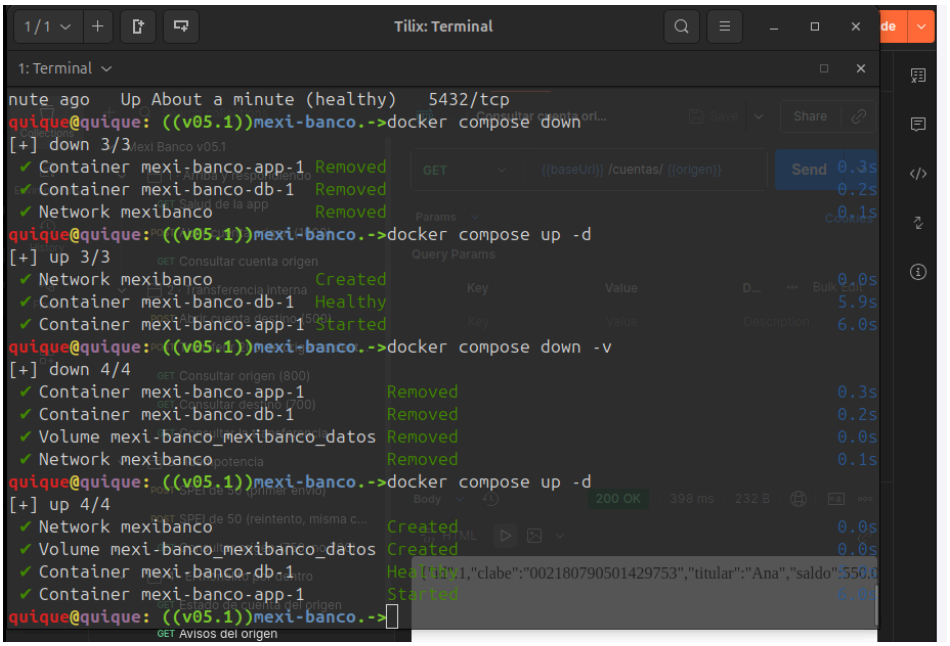


Para la prueba se vuelve a verificar un GET en la consulta de la cuenta origen, y se observa que los datos siguen intactos, ya que solo se apago el contenedor, el volumen que es el disco duro sigue sin cambios.

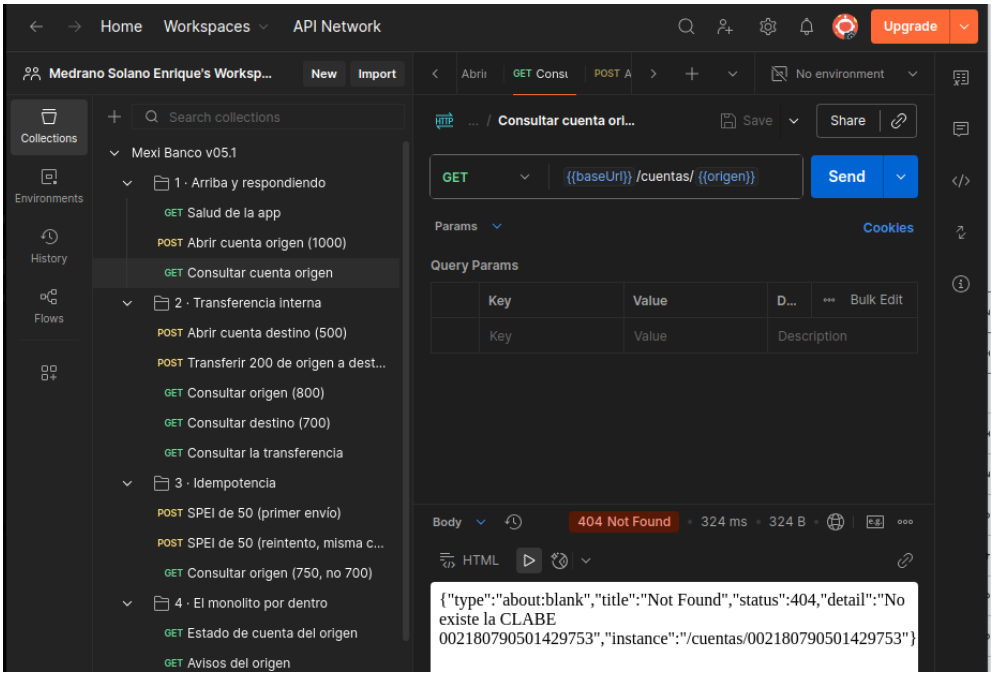


Ahora si se apaga el docker y el volumen y se vuelve a verificar el GET.

Laboratorio 5 - Sistemas Distribuidos
Medrano Solano Enrique
320152841



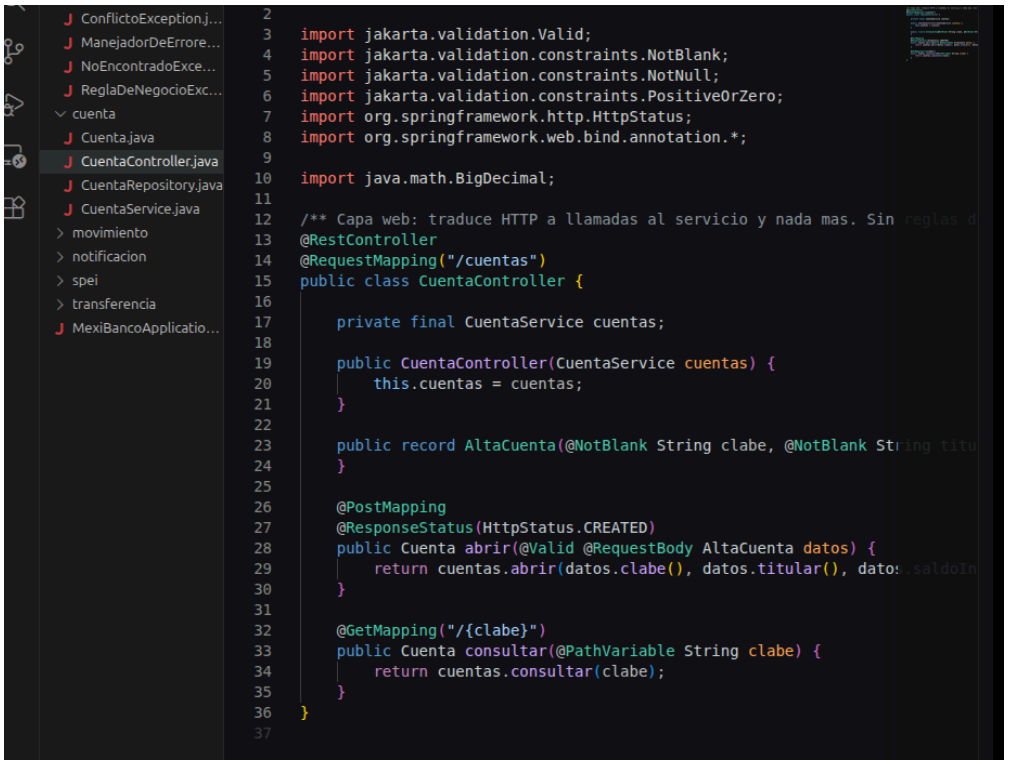
En este caso si manda error, ya que los datos del volumen fueron borrados, por lo que ya no encuentra la cuenta CLABE a la que se intenta acceder.



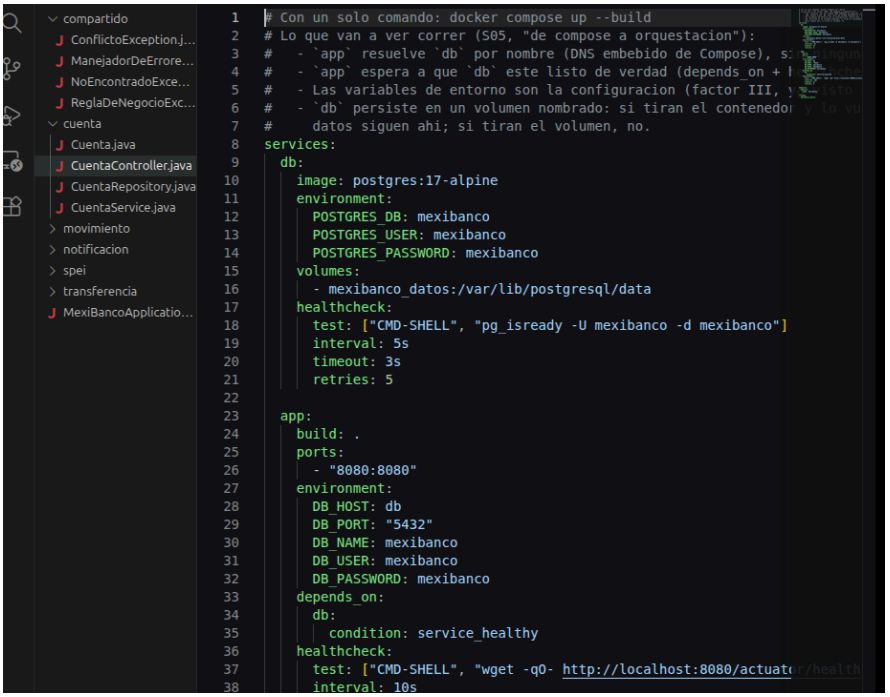
Se muestra la estructura del proyecto en la captura de CuentaController.java, organizada en paquetes como cuenta, movimiento, notificación, spei y transferencia. En el modulo de cuentas se aprecia la división por capas: Entidad (Cuenta.java), Controlador

Laboratorio 5 - Sistemas Distribuidos
Medrano Solano Enrique
320152841

(CuentaController.java), Repositorio (CuentaRepository.java) y Servicio (CuentaService.java).



El archivo de configuración de Docker Compose define dos servicios: una base de datos (db) usando la imagen de alpine 17 y la aplicación principal (app). Se observa que la aplicación (app) mapea el puerto 8080 y depende directamente del estado de salud de la base de datos mediante la instrucción depends_on con la condición service_healthy. Además, la base de datos utiliza un volumen nombrado (mexibanco_datos) para persistir la información.



Laboratorio 5 - Sistemas Distribuidos
Medrano Solano Enrique
320152841

C: > Users > enriq > Downloads > Dockerfile.txt > ...

```
# ---- ETAPA 1: CONSTRUCCION ----
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY mvnw .
COPY .mvn .mvn
RUN ./mvnw dependency:go-offline -B
COPY src ./src
RUN ./mvnw clean package -DskipTests

# ---- ETAPA 2: EJECUCION ----
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]

```

La etapa 1 utiliza una imagen base que incluye eclipse-temurin:21-jdk-alpine, el cual es el entorno de desarrollo completo para java. De modo que es el que prepara todo, copia el código fuentes y los archivos de configuración, descarga las dependencias necesarias y se compila todo el proyecto usando Maven para generar el archivo ejecutable.

Durante la etapa 2, utiliza una imagen más ligera que la primera en donde se incluye el entorno de ejecución de java eclipse-temurin:21-jre-alpine. Esta etapa copia únicamente el archivo **.jar** ya compilado desde la etapa anterior y se define el comando para el arranque en el puerto 8080